

Teaching LLMs to Use Tools

Adaptation of Agentic AI: Code
Reproduction & Autoresearch Teardown

Author: Akshata
Madavi

Course: Agentic
AI Adaptation

Base Source: Jiang et al. (2025) -
Adaptation of Agentic AI: A Survey
of Post-Training, Memory, and Skills

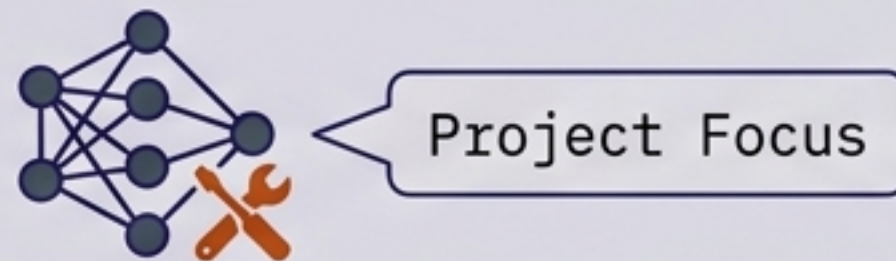
Status: 10
Iterations
Completed

Internal Agent Training

External Module Augmentation

[A1] SFT of the Agent

Fine-tuning the model directly on task demonstrations.



[T1] Tool/Skill Modules

Reusable plug-in tools the agent can call.

[A2] RL of the Agent

Reinforcement via rewards (RLHF, PPO, GRPO).

[T2] Memory / RAG

Retrieval-augmented generation from a memory store.

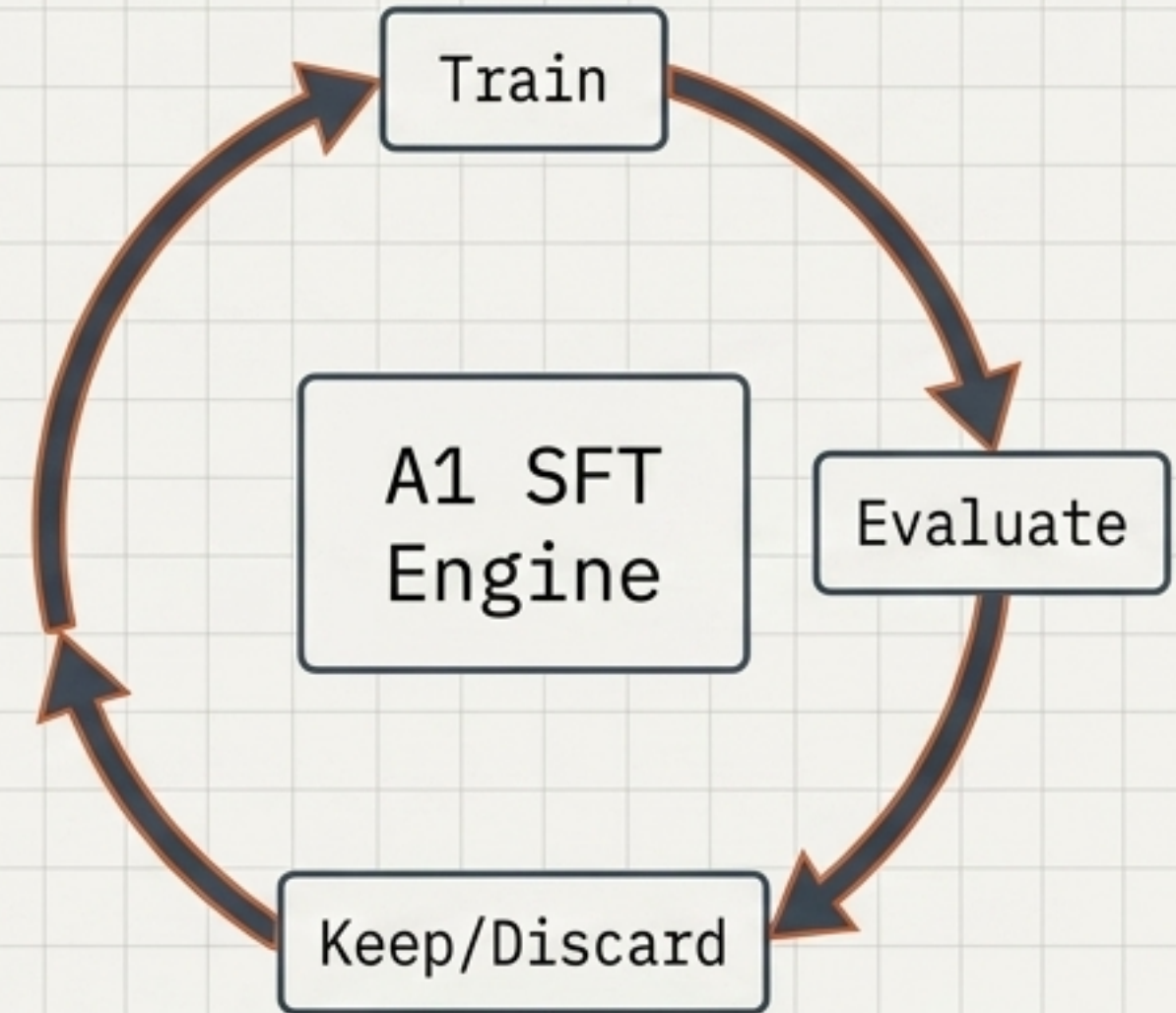
This project isolates the **A1 paradigm**, proving that Supervised Fine-Tuning alone can effectively teach an LLM rigid structural tool usage.

The Objective

Target: Supervised Fine-Tuning (SFT) of a local LLM agent to execute highly structured tool calls.

Challenge: Exacting schema adherence and precise argument extraction.

The Approach



Karpthy-Style Autoresearch Loop: Iterate over hyperparameters automatically, training on cloud hardware, and persistently keeping only mathematical improvements.

Execution Outputs: Wikipedia API & Calculator Tools

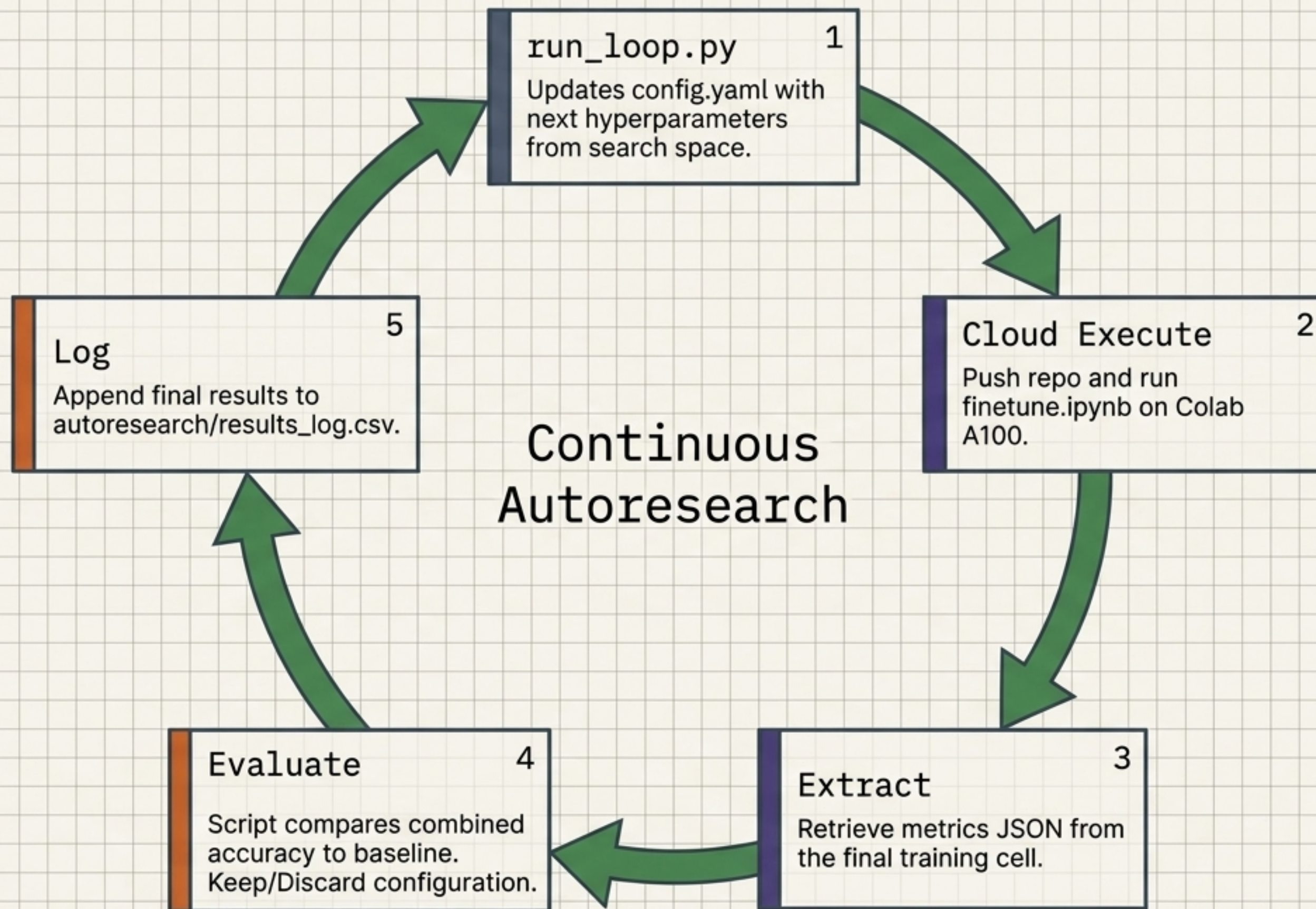
Base Model: meta-llama/Llama-3.2-3B-Instruct

Engine: LoRA / QLoRA via unsloth + SFTTrainer (trl)

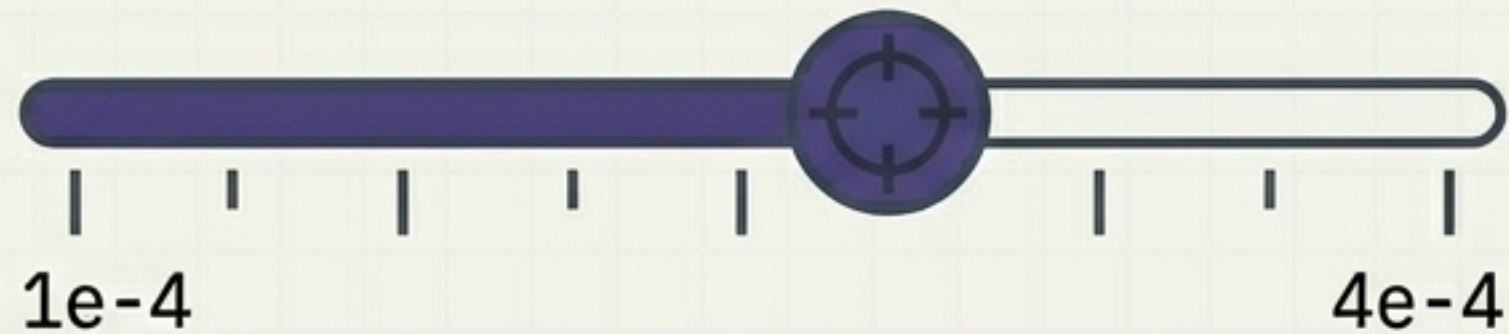
Data Source: Salesforce/xlam-function-calling-60k

Infrastructure: Google Colab Pro A100 (Cloud Training) -> Local Inference via HuggingFace transformers + peft

Auto-detects optimal hardware for local inference; bitsandbytes (4-bit quant) utilized exclusively on CUDA.



Learning Rate (lr)



LoRA Rank (lora_r)



Max Training Steps (max_steps)



Training Samples (train_samples)



The autoresearch loop navigated permutations of these four variables to identify the absolute ceiling of model performance.

Format Accuracy

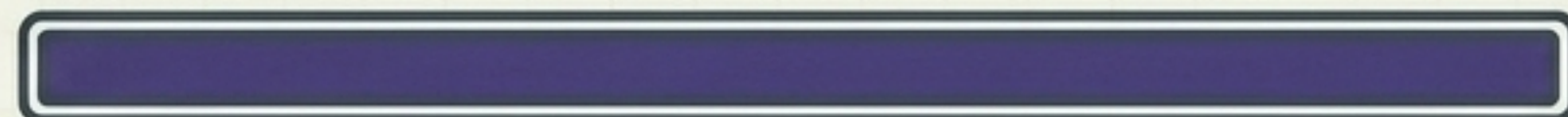
Does the model adhere to the strict output schema?



100%

Tool Name Accuracy

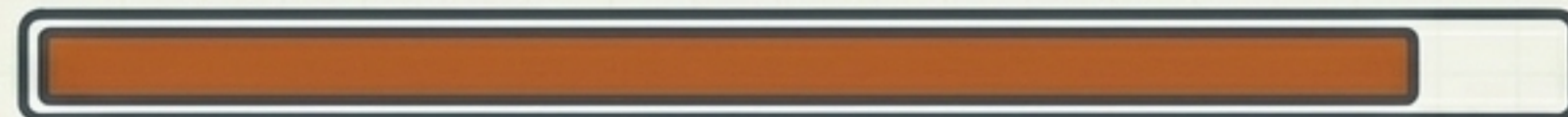
Did the model select the correct tool (e.g., Wikipedia vs. Calculator) for the given prompt?



100%

Argument Accuracy

Exact value matching. Did the model extract and format the precise variables required for the tool?



90%

Format + Tool Name + Arguments = Weighted Combined Accuracy

97.20%

Peak Combined Accuracy (Achieved in Iteration 3)

The Winning Recipe

`lora_r: 16`

`lr: 2e-4`

`max_steps: 500`

`train_samples: 5000`

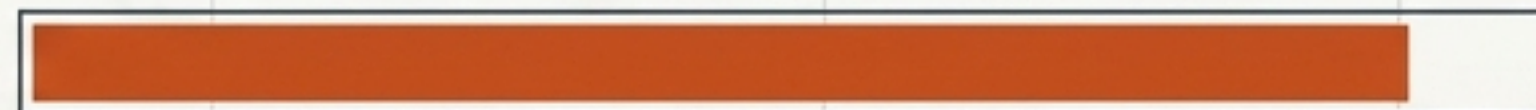
Format Accuracy: 100.00% 100%



Tool Name Accuracy: 100.00% 100%



Argument Accuracy: 91.50% 91.50%



Percentage



Format and tool-name accuracy saturate near 100% across all configurations. The model easily learns the output schema.

Argument extraction is the universal bottleneck. Exact value matching is fundamentally harder for the LLM than structural adherence.

Diminishing Returns: Why did failed experiments not beat the baseline?

The Compute/Data Trap

(Iter 8, 9, 10)

Doubling training data (10k) or doubling training steps (1000) yielded 0% improvement over the 5k/500-step baseline.

The Learning Rate Penalty

(Iter 4)

Pushing `lr` higher to `4e-4` actively damaged argument accuracy (dropped to 90.00%).

The Architecture Penalty

(Iter 6)

Using a larger `lora_r` of 32, but with minimal target modules, resulted in the worst argument accuracy of the run (88.50%).

Project Deliverables

-  Article: Medium - Teaching an LLM to Use Tools
-  Deck: Slideshare - Visual summary
-  Video: YouTube - Full code walkthrough
-  Code: HuggingFace Adapter (`adapter_repo` via `config.yaml`)

[Process Terminated. Target A1 SFT
successfully reproduced.]